

Requisitos do Sistema — RadarTorres

1 Introdução

A Engenharia de Requisitos é a disciplina da Engenharia de Software responsável por descobrir, analisar, documentar e verificar os serviços que um sistema deve prestar e as restrições sob as quais deve operar [REFERÊNCIA BIBLIOGRÁFICA A INSERIR]. Requisitos de software podem ser entendidos como as condições ou capacidades que um sistema precisa apresentar para satisfazer as necessidades de seus usuários, sejam elas expressas formalmente por um cliente, sejam inferidas do contexto de uso pretendido.

O levantamento de requisitos é a etapa em que essas necessidades são identificadas junto aos usuários e demais interessados, antes de qualquer decisão de projeto ou implementação; sua qualidade determina diretamente a qualidade de todo o desenvolvimento subsequente, porque qualquer decisão arquitetural, de interface ou de dados parte do que foi levantado nessa etapa. A especificação, por sua vez, é o registro formal e verificável desses requisitos — o contrato entre o que os usuários esperam e o que o sistema efetivamente entrega, servindo tanto de guia para o desenvolvimento quanto de critério objetivo para validar se o sistema construído atende ao que foi solicitado.

Requisitos mal definidos — ambíguos, incompletos, contraditórios entre si ou desalinhados das reais expectativas dos usuários — tendem a se manifestar tardiamente, quando corrigi-los já é mais custoso: funcionalidades implementadas que não atendem à necessidade real, retrabalho, divergência entre o que a documentação descreve e o que o software efetivamente faz. Por essa razão, este documento distingue deliberadamente, para o RadarTorres, o que é comportamento observável do produto (requisito), o que é decisão interna de construção (não é requisito) e o que é funcionalidade ainda não implementada (não deve ser apresentada como requisito atendido).

2 Requisitos Funcionais e Não Funcionais

A literatura de Engenharia de Requisitos distingue, tradicionalmente, dois grandes tipos de requisito.

2.1 Requisito Funcional

Comportamento ou capacidade que o sistema deve fornecer.

Um requisito funcional descreve uma ação concreta, disparada por um ator (um usuário ou outro sistema), que produz um resultado observável. Pode-se testar sua satisfação de forma direta: dado um estímulo, o sistema produz (ou não) o comportamento esperado.

2.2 Requisito Não Funcional

Restrição ou atributo de qualidade relacionado a desempenho, segurança, usabilidade, confiabilidade, compatibilidade, portabilidade etc.

Um requisito não funcional não descreve uma ação isolada, mas uma característica que atravessa várias (ou todas as) funcionalidades do sistema — o quão bem, quão rápido, quão seguro ou quão disponível o sistema se comporta ao realizar o que os requisitos funcionais especificam.

Este documento evita deliberadamente misturar decisões internas de arquitetura (por exemplo, a escolha de um padrão de projeto, de uma biblioteca ou de uma forma de persistência) com requisitos do produto: uma decisão de implementação só se torna requisito quando é, ela própria, uma característica observável ou exigida externamente — do contrário, é tratada à parte, na Seção 5.

3 Requisitos Funcionais

Levantados a partir do código-fonte do RadarTorres e da documentação técnica do projeto (Docs/Diagramas_e_requisitos/Requisitos_Funcionais.md, já revisada para separar produto, arquitetura e limitações — ver Docs/LOG_SOLICITACOES.md para o histórico dessa revisão). Os identificadores RF07, RF25 e RF32 foram mantidos na numeração, marcados como removidos da especificação funcional, para preservar a rastreabilidade histórica com versões anteriores da documentação — sem deixar nenhuma lacuna silenciosa na sequência de IDs.

3.1 Tabela resumida

ID	Requisito Funcional	Prioridade	Status
RF01	Comunicação serial com o Arduino	Alta	Implementado
RF02	Recepção e validação de leituras de alvo	Alta	Implementado
RF03	Rastreamento de alvos em tempo real	Alta	Implementado
RF04	Exibição do radar circular em tempo real	Alta	Implementado
RF05	Seleção automática de torre	Alta	Implementado
RF06	Acionamento demonstrativo automático	Alta	Implementado

ID	Requisito Funcional	Prioridade	Status
RF07	<i>Removido da especificação funcional</i>	—	Removido
RF08	Modos de operação do sistema	Alta	Parcial
RF09	Autenticação multiusuário	Alta	Implementado
RF10	Controle de acesso por perfil	Alta	Implementado
RF11	Troca de senha	Média	Implementado
RF12	Registro histórico de objetos detectados	Média	Implementado
RF13	Visualização de objetos detectados em tabela	Média	Implementado
RF14	Exportação de objetos detectados	Baixa	Implementado
RF15	Importação de objetos detectados	Baixa	Implementado
RF16	Auditoria de ações realizadas	Média	Parcial
RF17	Auditoria de alterações de modo	Média	Parcial
RF18	Gerenciamento de usuários	Média	Planejado
RF19	Preferências de usuário	Baixa	Implementado
RF20	Troca de idioma da interface	Baixa	Implementado
RF21	Troca de tema visual	Baixa	Implementado
RF22	Abertura de chamado de ajuda/suporte	Baixa	Implementado
RF23	Tratamento administrativo de chamados de ajuda	Baixa	Planejado
RF24	Painel principal com cards personalizáveis	Baixa	Implementado
RF25	<i>Removido — consolidado em RNF11</i>	—	Removido
RF26	Console de eventos fixável na lateral	Baixa	Implementado
RF27	Gestão de zonas mortas	Alta	Implementado
RF28	Deteção do Arduino CLI	Média	Implementado
RF29	Compilação de sketch Arduino pela interface	Média	Implementado

ID	Requisito Funcional	Prioridade	Status
RF30	Monitor serial pela aba Configurações do Arduino	Média	Implementado
RF31	Persistência de preferências da aba Arduino	Baixa	Implementado
RF32	<i>Removido — consolidado em RNF17/RNF18</i>	—	Removido

3.2 Especificação individual

3.2.1 RF01 — Comunicação serial com o Arduino

Descrição: o sistema deve conectar a uma porta serial USB, listar as portas disponíveis, enviar e receber mensagens em um protocolo textual definido, e detectar perda de conexão por watchdog. **Ator(es):** Usuário (operador), Arduino. **Prioridade:** Alta. **Status:** Implementado.

3.2.2 RF02 — Recepção e validação de leituras de alvo

Descrição: o sistema deve interpretar mensagens de leitura de alvo, validar seus campos e registrar leituras inválidas como erro, sem interromper a aplicação. **Ator(es):** Arduino, Sistema. **Prioridade:** Alta. **Status:** Implementado.

3.2.3 RF03 — Rastreamento de alvos em tempo real

Descrição: o sistema deve criar um novo alvo a cada leitura válida com identificador inédito, atualizar o alvo existente quando o identificador já for conhecido, e expirar automaticamente alvos sem leitura recente após um tempo configurável. **Ator(es):** Sistema. **Prioridade:** Alta. **Status:** Implementado.

3.2.4 RF04 — Exibição do radar circular em tempo real

Descrição: o sistema deve exibir os alvos ativos em um radar circular dividido em quatro quadrantes, reposicionando-os continuamente conforme novas leituras chegam. **Ator(es):** Usuário. **Prioridade:** Alta. **Status:** Implementado.

3.2.5 RF05 — Seleção automática de torre

Descrição: o sistema deve selecionar, entre as torres configuradas, a mais adequada para cada alvo detectado, considerando quadrante preferencial e distância, e respeitando zonas mortas ativas. **Ator(es):** Sistema. **Prioridade:** Alta. **Status:** Implementado.

3.2.6 RF06 — Acionamento demonstrativo automático

Descrição: no modo Vermelho, o sistema deve realizar automaticamente o acionamento demonstrativo (laser de baixa potência, LED ou simulação — nunca armamento real) sobre o alvo acompanhado pela torre selecionada, respeitando as regras de segurança (distância mínima e zonas mortas ativas). No modo Amarelo, as torres devem apenas acompanhar os alvos, sem realizar acionamento. Não existe acionamento manual: toda tentativa é decidida pelo sistema, e cada tentativa (autorizada, bloqueada ou com erro) deve ser registrada em auditoria. **Ator(es):** Sistema. **Prioridade:** Alta. **Status:** Implementado (*ver divergência técnica na Seção 6 — o código expõe adicionalmente um caminho de acionamento manual não previsto nesta especificação*).

3.2.7 RF07 — Removido da especificação funcional

O modo de simulação sem hardware existe exclusivamente para desenvolvimento, testes e demonstração do sistema sem um Arduino conectado — não é uma função operacional entregue ao usuário final do produto, e sim um recurso de apoio a desenvolvimento e demonstração. Por isso não integra a lista oficial de requisitos funcionais; permanece documentado tecnicamente em Docs/DOCUMENTACAO_TECNICA.md e no README.md do projeto.

3.2.8 RF08 — Modos de operação do sistema

Descrição: o sistema deve operar em exatamente três estados: **Verde** (ligado, porém sem operação funcional de rastreamento ou acionamento), **Amarelo** (detecta e rastreia alvos, e as torres acompanham automaticamente o alvo selecionado, sem acionar) e **Vermelho** (detecta, rastreia, acompanha automaticamente, e realiza o acionamento demonstrativo automaticamente quando autorizado pelas regras de segurança de RF06). A troca de estado deve exigir confirmação do usuário antes de aplicar e ser registrada em auditoria. **Ator(es):** Usuário. **Prioridade:** Alta. **Status:** Parcial (*ver divergência técnica na Seção 6 — o código ainda usa uma nomenclatura de modos anterior a esta especificação*).

3.2.9 RF09 — Autenticação multiusuário

Descrição: o sistema deve exigir login (usuário/senha) independente da conta do sistema operacional, com senha protegida por hash e salt por usuário. **Ator(es):** Usuário. **Prioridade:** Alta. **Status:** Implementado.

3.2.10 RF10 — Controle de acesso por perfil

Descrição: o sistema deve restringir a visibilidade de menu e a execução de ações conforme o perfil do usuário logado (Administrador, Operador, Visualizador) — o perfil Visualizador deve ser somente consulta. **Ator(es):** Sistema, Usuário. **Prioridade:** Alta. **Status:** Implementado.

3.2.11 RF11 — Troca de senha

Descrição: o usuário logado deve poder alterar sua própria senha, mediante validação da senha atual antes da troca. **Ator(es):** Usuário. **Prioridade:** Média. **Status:** Implementado.

3.2.12 RF12 — Registro histórico de objetos detectados

Descrição: a primeira detecção de cada alvo deve ser gravada como um registro histórico, com posição, quadrante, horário e dispositivo de origem. **Ator(es):** Sistema. **Prioridade:** Média. **Status:** Implementado.

3.2.13 RF13 — Visualização de objetos detectados em tabela

Descrição: o sistema deve exibir o histórico de detecções em uma tela de tabela dedicada. **Ator(es):** Usuário. **Prioridade:** Média. **Status:** Implementado.

3.2.14 RF14 — Exportação de objetos detectados

Descrição: o usuário deve poder exportar a lista de objetos detectados nos formatos CSV, XML ou PDF, disponível para qualquer perfil autenticado. **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.15 RF15 — Importação de objetos detectados

Descrição: o usuário deve poder importar registros de um arquivo CSV ou XML no mesmo formato da exportação; restrito a perfis que podem executar ações (Visualizador não pode importar). **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.16 RF16 — Auditoria de ações realizadas

Descrição: cada tentativa de acionamento deve ser gravada para consulta posterior, exclusivamente em modo de inserção. **Ator(es):** Sistema, Usuário (consulta). **Prioridade:** Média. **Status:** Parcial — o registro já é gravado automaticamente a cada tentativa de acionamento; a tela dedicada de consulta ainda não foi implementada.

3.2.17 RF17 — Auditoria de alterações de modo

Descrição: cada troca de modo de operação (RF08) deve ser gravada para consulta posterior. **Ator(es):** Sistema, Usuário (consulta). **Prioridade:** Média. **Status:** Parcial — o registro já é gravado automaticamente a cada troca de modo; a tela dedicada de consulta ainda não foi implementada.

3.2.18 RF18 — Gerenciamento de usuários

Descrição: um Administrador deve poder criar, editar e inativar contas de usuário. **Ator(es):** Usuário (Administrador). **Prioridade:** Média. **Status:** Planejado — a base de dados e a restrição de permissão já existem; nenhuma tela consome esse recurso hoje.

3.2.19 RF19 — Preferências de usuário

Descrição: o sistema deve salvar e restaurar, por usuário, o idioma preferido, o tema e o estado da barra lateral. **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.20 RF20 — Troca de idioma da interface

Descrição: o usuário deve poder alterar o idioma da interface entre português (Brasil) e inglês (EUA). **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.21 RF21 — Troca de tema visual

Descrição: o usuário deve poder selecionar o tema visual da interface entre claro, escuro ou acompanhar a configuração do sistema operacional. **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.22 RF22 — Abertura de chamado de ajuda/suporte

Descrição: o usuário deve poder abrir um chamado de suporte (título, descrição, categoria, módulo relacionado, mensagem de erro), com usuário e data preenchidos automaticamente. **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.23 RF23 — Tratamento administrativo de chamados de ajuda

Descrição: um Administrador deve poder consultar os chamados abertos e definir situação e resposta para cada um. **Ator(es):** Usuário (Administrador). **Prioridade:** Baixa. **Status:** Planejado — a base de dados já expõe a operação de atualização; nenhuma tela consome esse recurso hoje.

3.2.24 RF24 — Painel principal com cards personalizáveis

Descrição: o usuário deve poder arrastar e redimensionar cards de indicadores no painel principal; posição, tamanho, visibilidade e ordem devem ser salvos por usuário. **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.25 RF25 — Removido da especificação funcional

A responsividade do layout a mudanças de resolução/tamanho de janela é uma característica de qualidade (usabilidade/compatibilidade), não uma função acionada por um ator. Consolidado em RNF11, sem duplicar aqui.

3.2.26 RF26 — Console de eventos fixável na lateral

Descrição: o usuário deve poder fixar o console de eventos na borda direita da tela de Monitoramento, com o estado persistido. **Ator(es):** Usuário. **Prioridade:** Baixa. **Status:** Implementado.

3.2.27 RF27 — Gestão de zonas mortas

Descrição: um Administrador deve poder criar, ativar/desativar e remover zonas onde alvos não recebem torre nem podem ser acionados, embora continuem visíveis/rastreados; demais perfis devem visualizar a lista somente leitura. **Ator(es):** Usuário (Administrador), Sistema. **Prioridade:** Alta. **Status:** Implementado.

3.2.28 RF28 — Detecção do Arduino CLI

Descrição: o sistema deve localizar o executável do Arduino CLI no computador, sem baixar nada automaticamente, e exibir a versão detectada. **Ator(es):** Usuário, Sistema. **Prioridade:** Média. **Status:** Implementado.

3.2.29 RF29 — Compilação de sketch Arduino pela interface

Descrição: o usuário deve poder selecionar um sketch e uma placa, compilar via Arduino CLI como processo assíncrono e cancelável, acompanhando a saída em tempo real. **Ator(es):** Usuário, Sistema. **Prioridade:** Média. **Status:** Implementado.

3.2.30 RF30 — Monitor serial pela aba Configurações do Arduino

Descrição: o usuário deve poder acompanhar mensagens da porta serial diretamente nesta aba, reutilizando a mesma conexão da tela de Monitoramento. **Ator(es):** Usuário, Sistema, Arduino. **Prioridade:** Média. **Status:** Implementado.

3.2.31 RF31 — Persistência de preferências da aba Arduino

Descrição: caminho do CLI, último sketch, placa, porta/ baud e preferências do console devem ser salvos e restaurados entre sessões. **Ator(es):** Sistema. **Prioridade:** Baixa. **Status:** Implementado.

3.2.32 RF32 — Removido da especificação funcional

Instalação, empacotamento self-contained e preservação de configurações em upgrade são características de portabilidade/instalação/manutenibilidade, não uma função acionada por um ator do sistema em operação. Consolidado em RNF17 e RNF18, sem duplicar aqui.

4 Requisitos Não Funcionais

Organizados por categoria. Requisitos que descreviam decisão interna de implementação em vez de característica de qualidade do produto (antigos RNF20–RNF23 e RNF30) foram retirados desta lista e tratados na Seção 5; o que descrevia limitação não implementada (antigo RNF29) foi tratado na Seção 6 — em ambos os casos, o ID é referenciado, não reaproveitado para outro conteúdo.

4.1 Tabela resumida

ID	Requisito Não Funcional	Categoria	Prioridade	Status
RNF01	Redesenho do radar em ciclo curto e configurável	Desempenho	Média	Atendido
RNF02	Leitura serial não bloqueante	Desempenho / Confiabilidade	Alta	Atendido
RNF03	Degradação graciosa de falhas de hardware/serial	Confiabilidade	Alta	Atendido
RNF04	Distância mínima de segurança obrigatória	Segurança	Alta	Atendido
RNF05	Acionamento é só demonstrativo, nunca armamento real	Segurança	Alta	Atendido
RNF06	Senha de usuário nunca em texto puro	Segurança	Alta	Atendido
RNF07	Controle de acesso centralizado por perfil	Segurança	Alta	Atendido
RNF08	Zonas mortas só editáveis por Administrador	Segurança	Alta	Atendido
RNF09	Montagem segura de comandos de processo externo	Segurança	Média	Atendido
RNF10	Arduino CLI nunca baixado automaticamente	Segurança	Média	Atendido

ID	Requisito Não Funcional	Categoria	Prioridade	Status
RNF11	Interface responsiva a mudanças de resolução/janela	Usabilidade / Compatibilidade	Média	Atendido
RNF12	Troca de idioma sem reinício	Usabilidade	Baixa	Atendido
RNF13	Consistência visual entre temas	Usabilidade	Baixa	Atendido
RNF14	Limite de linhas nos consoles	Desempenho / Confiabilidade	Média	Atendido
RNF15	Mutação de coleções de UI sempre na thread de interface	Confiabilidade	Alta	Atendido
RNF16	Compilação do sketch não bloqueia a interface	Desempenho / Usabilidade	Média	Atendido
RNF17	Runtime portátil, sem dependência externa	Portabilidade / Instalação	Média	Atendido
RNF18	Upgrade preserva configurações do usuário	Manutenibilidade / Instalação	Média	Atendido
RNF19	Persistência gravável sem privilégio de administrador	Manutenibilidade	Média	Atendido
RNF24	Cobertura de testes automatizados dos componentes críticos	Confiabilidade	Média	Parcial
RNF25	Baud rate configurável	Comunicação	Baixa	Atendido
RNF26	Codificação e terminador de linha fixos no protocolo	Comunicação	Baixa	Atendido
RNF27	Plataforma-alvo Windows Desktop	Compatibilidade	Alta	Atendido
RNF28	Tolerância a mensagens malformadas do Arduino	Confiabilidade	Média	Atendido

4.2 Observações de reformulação

RNF16. Para não duplicar RF29 (que já descreve a funcionalidade de compilar/cancelar pela interface), este requisito não funcional foi restrito à propriedade de qualidade associada: *a execução da compilação não deve bloquear a interface do usuário, e sua interrupção deve ocorrer de maneira controlada.*

RNF12 e RNF13. Pelo mesmo princípio, foram restritos à qualidade da troca (tempo de execução, sem reinício; consistência visual entre temas), deixando a existência da funcionalidade em si — o usuário poder escolher idioma/tema — a cargo de RF20/RF21.

RNF24. Não é utilizada a formulação "o sistema possui 21 testes" como requisito — quantidade de testes é evidência de estado atual, não um requisito em si. A formulação adotada é: *os componentes críticos de comunicação, integração com o Arduino CLI e persistência devem possuir testes automatizados que validem os principais cenários de sucesso e falha.* Evidência atual: 21 testes xUnit aprovados, concentrados na aba de configuração do Arduino — os demais componentes citados (comunicação serial de leitura de alvos, rastreamento, seleção de torre, persistência CSV) ainda não têm teste automatizado, por isso o status é Parcial, não Atendido.

5 Decisões e Restrições Arquiteturais

Esta seção reúne escolhas internas de construção do RadarTorres que não são requisitos do produto — não descrevem algo que um usuário aciona ou percebe como característica de qualidade, mas decisões de engenharia sobre como o software foi construído. A distinção é relevante porque trocar qualquer uma dessas decisões (por exemplo, substituir a persistência CSV por um banco de dados relacional) não mudaria nada que um usuário observasse funcionando — mudaria apenas o quão fácil é manter e evoluir o código, uma preocupação de engenharia, não do produto entregue.

Decisões comprovadas no código-fonte:

- **Linguagem e runtime:** C# 13 sobre .NET 9, com publicação self-contained para Windows Desktop (64-bit).
- **Interface:** Windows Presentation Foundation (WPF).
- **Arquitetura de apresentação:** MVVM (*Model-View-ViewModel*), implementado manualmente (`ViewModelBase`, `RelayCommand`) em vez de um framework externo — decisão didática, para manter o mecanismo de binding inteiramente explicável na defesa do trabalho.
- **Injeção de dependência:** `Microsoft.Extensions.DependencyInjection`, com a composição de serviços centralizada em `App.xaml.cs`.
- **Comunicação com hardware:** `System.IO.Ports`, com o protocolo serial inteiramente interpretado e montado por um único componente (`SerialProtocolParser`), evitando strings de protocolo soltas em outras classes.

- **Persistência:** atualmente em arquivos CSV locais, por decisão explícita registrada no projeto; cada entidade de domínio já possui uma interface de repositório dedicada, preparada para uma futura migração a um banco relacional (SQLite/EF Core) sem exigir alteração nas camadas de `ViewModel/Service`.
- **Separação de camadas:** nenhuma classe de `Services/Models` depende de WPF — a interface gráfica depende dos serviços através de interfaces (`I*Service`), nunca o inverso.

Uma decisão adicional, de natureza mais próxima de interação/UX do que de arquitetura de software, também foi identificada e separada dos requisitos de qualidade: o painel principal recusa o gesto de arraste/redimensionamento de um card que colidiria com outro, em vez de reorganizar os demais cards automaticamente. Trata-se de uma escolha de comportamento de interface — mais previsível para o usuário — sem uma exigência de negócio explícita por trás dela, por isso tratada aqui como decisão de design, não como requisito não funcional.

A relação completa dessas decisões com o arquivo/classe correspondente está em `Docs/Diagramas_e_requisitos/Decisoes_Arquiteturais.md`, que preserva a referência aos IDs de RNF originalmente atribuídos a cada uma antes desta reclassificação (RNF20–RNF23 e RNF30).

6 Limitações atuais

Nem toda lacuna entre a especificação e o sistema real é uma decisão de arquitetura: parte dela é, simplesmente, funcionalidade ainda não implementada. Registrar essas limitações separadamente evita que uma tela ainda incompleta seja lida como um requisito atendido.

- **Telas de consulta/gestão ainda em construção.** Cinco itens do menu de navegação levam a uma tela ainda não implementada: Ações Realizadas e Histórico de Modos (RF16/RF17 — o registro de dados já funciona, falta a tela de consulta), Usuários e gestão de Chamados de Ajuda (RF18/RF23 — nem o registro tem interface de gestão) e Configurações (nunca chegou a ser escopada).
- **Reconexão serial automática não implementada.** Após uma queda de conexão, a reconexão é manual; existe um campo de configuração (`ReconnectAttempts`) que expressa a intenção do requisito, mas a lógica de retry automático ainda não foi construída.
- **Persistência ainda em CSV.** Como descrito na Seção 5, a persistência em arquivos CSV é a implementação atual, com a interface de repositório já preparada para uma futura migração a banco relacional — migração que, até o momento, não foi realizada.

- **Upload/gravação de firmware no Arduino ainda não implementado.** A aba de Configurações do Arduino hoje só compila o sketch (RF29); gravar o binário compilado na placa é o próximo passo natural dessa funcionalidade, mas depende de decisão e implementação futuras.
- **Bugs conhecidos, não corrigidos.** Já catalogados tecnicamente em Docs/DOCUMENTACAO_TECNICA.md — um tratador de exceções sem trava de reentrância que pode, em cenários específicos, encerrar o processo sem exibir mensagem ao usuário; e uma falha de renderização de texto observada apenas em ambiente de automação, ainda não confirmada em uso interativo normal.
- **Divergência técnica entre RF06/RF08 e o código atual.** O enumerador que representa o modo de operação do sistema (`SystemMode`) ainda utiliza seis valores herdados de uma nomenclatura anterior (`Off`, `LocationOnly`, `LocationAutoTower`, `LocationAutoFire`, `Maintenance`, `Emergency`), não os três estados conceituais Verde/Amarelo/Vermelho definidos em RF08; e o código ainda expõe um comando de acionamento manual, sem checagem do modo de operação antes de autorizar o disparo — o que hoje torna tecnicamente possível acionar manualmente mesmo em um modo que, pela especificação revisada, deveria apenas acompanhar o alvo. Nenhuma dessas divergências foi corrigida no código como parte deste trabalho documental; ambas ficam registradas para uma tarefa de implementação futura, com autorização explícita.

7 Relação entre requisitos e UML

Os requisitos funcionais e não funcionais especificados neste documento não são um exercício isolado: serviram diretamente de base para a modelagem UML apresentada em Docs/TCC/UML_RadarTorres.md. O diagrama de casos de uso deriva, requisito a requisito, dos RFs válidos listados na Seção 3 — cada caso de uso do diagrama está ligado a pelo menos um requisito funcional real, e nenhum foi incluído sem essa correspondência. O diagrama de classes representa as entidades e os serviços que sustentam esses mesmos requisitos (o alvo rastreado, a torre, a zona morta, os registros de auditoria). O diagrama de implantação decorre diretamente dos requisitos de comunicação com o Arduino (RF01, RF02) e das restrições de plataforma registradas como não funcionais (RNF27). Manter essa correspondência explícita entre requisito e diagrama é o que garante coerência entre a especificação do sistema e o projeto técnico que a implementa.

8 Matriz resumida

Requisito	Categoria	Prioridade	Status
RF01	Funcional	Alta	Implementado

Requisito	Categoria	Prioridade	Status
RF02	Funcional	Alta	Implementado
RF03	Funcional	Alta	Implementado
RF04	Funcional	Alta	Implementado
RF05	Funcional	Alta	Implementado
RF06	Funcional	Alta	Implementado
RF07	Funcional	—	Removido
RF08	Funcional	Alta	Parcial
RF09	Funcional	Alta	Implementado
RF10	Funcional	Alta	Implementado
RF11	Funcional	Média	Implementado
RF12	Funcional	Média	Implementado
RF13	Funcional	Média	Implementado
RF14	Funcional	Baixa	Implementado
RF15	Funcional	Baixa	Implementado
RF16	Funcional	Média	Parcial
RF17	Funcional	Média	Parcial
RF18	Funcional	Média	Planejado
RF19	Funcional	Baixa	Implementado
RF20	Funcional	Baixa	Implementado
RF21	Funcional	Baixa	Implementado
RF22	Funcional	Baixa	Implementado
RF23	Funcional	Baixa	Planejado
RF24	Funcional	Baixa	Implementado
RF25	Funcional	—	Removido
RF26	Funcional	Baixa	Implementado
RF27	Funcional	Alta	Implementado
RF28	Funcional	Média	Implementado
RF29	Funcional	Média	Implementado
RF30	Funcional	Média	Implementado
RF31	Funcional	Baixa	Implementado
RF32	Funcional	—	Removido
RNF01	Desempenho	Média	Atendido
RNF02	Desempenho / Confiabilidade	Alta	Atendido
RNF03	Confiabilidade	Alta	Atendido
RNF04	Segurança	Alta	Atendido
RNF05	Segurança	Alta	Atendido
RNF06	Segurança	Alta	Atendido
RNF07	Segurança	Alta	Atendido
RNF08	Segurança	Alta	Atendido
RNF09	Segurança	Média	Atendido
RNF10	Segurança	Média	Atendido

Requisito	Categoria	Prioridade	Status
RNF11	Usabilidade / Compatibilidade	Média	Atendido
RNF12	Usabilidade	Baixa	Atendido
RNF13	Usabilidade	Baixa	Atendido
RNF14	Desempenho / Confiabilidade	Média	Atendido
RNF15	Confiabilidade	Alta	Atendido
RNF16	Desempenho / Usabilidade	Média	Atendido
RNF17	Portabilidade / Instalação	Média	Atendido
RNF18	Manutenibilidade / Instalação	Média	Atendido
RNF19	Manutenibilidade	Média	Atendido
RNF24	Confiabilidade	Média	Parcial
RNF25	Comunicação	Baixa	Atendido
RNF26	Comunicação	Baixa	Atendido
RNF27	Compatibilidade	Alta	Atendido
RNF28	Confiabilidade	Média	Atendido

A evidência técnica detalhada (arquivo/classe/função por requisito) permanece exclusivamente em `Docs/Diagramas_e_requisitos/Matriz_de_Rastreabilidade.md`, não duplicada nesta matriz resumida.